



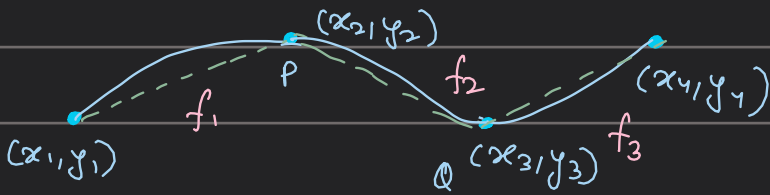
Driverless-ppc (Path planing & control)

Path Planning:

Controls $\begin{cases} \text{steering} \\ \text{throttle} \\ \text{brake} \end{cases}$

Meet-1

Quadratic Interpolation:



the shortest path is the st. line but we don't want sharp turns

f_1, f_2, f_3 : Quadratic

$$\begin{aligned} P: f_1(x_2, y_2) = f_2(x_2, y_2) &\Rightarrow f_1'(x_2, y_2) = f_2'(x_2, y_2) \\ Q: f_2(x_3, y_3) = f_3(x_3, y_3) &\Rightarrow f_2'(x_3, y_3) = f_3'(x_3, y_3) \end{aligned} \left. \begin{array}{l} \text{cond'n:} \\ \text{continuity} \\ \text{diffble} \end{array} \right\}$$

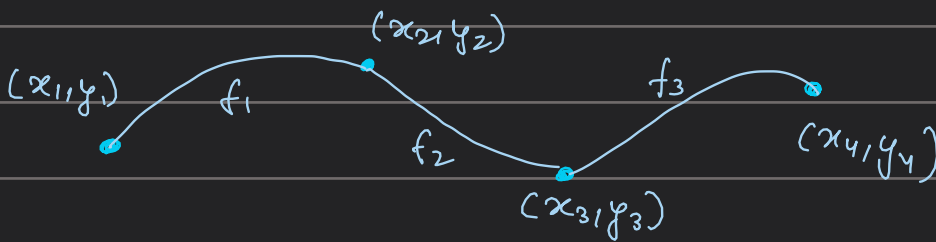
$$f_1: a_1x^2 + b_1x + c_1, \quad f_2: a_2x^2 + b_2x + c_2, \quad f_3: a_3x^2 + b_3x + c_3$$

we are short of the no. of equations $\Rightarrow$ assume $f_1(x_1, y_1) = f_3(x_4, y_4) = 0$

But this is not a very good assumption

so we use Cubic interpolation

Cubic Interpolation:



f_1, f_2, f_3 : Cubic
 $\downarrow$
continuous & diffble

$$\Rightarrow f_1(x_2, y_2) = f_2(x_2, y_2) = f_1'(x_2, y_2) = f_2'(x_2, y_2) = f_1''(x_2, y_2) = f_2''(x_2, y_2)$$

$$\Rightarrow f_2(x_3, y_3) = f_3(x_3, y_3) = f_2'(x_3, y_3) = f_3'(x_3, y_3) = f_2''(x_3, y_3) = f_3''(x_3, y_3)$$

$$f_1: a_1x^3 + b_1x^2 + c_1x + d_1$$

$$f_2: a_2x^3 + b_2x^2 + c_2x + d_2$$

$$f_3: a_3x^3 + b_3x^2 + c_3x + d_3$$

We let $f_1(x_1, y_1)$ & $f_3(x_4, y_4)$ to be equal to some const.

Q1) Why we don't create a polynomial which passes through all the points.

Our aim is to create a polynomial like this, but since there can only be one polynomial which passes through all the 7 points it may be like this & will hamper the shape of the track.

that's why we break the track into smaller points to ensure its shape.

Interpolation of a closed path.

Meet-2

parametrise the curve as a fn of distance from start

Optimisation of the curvature

we have to optimise the curvature by $(|curvature at a|)^2 + \dots$

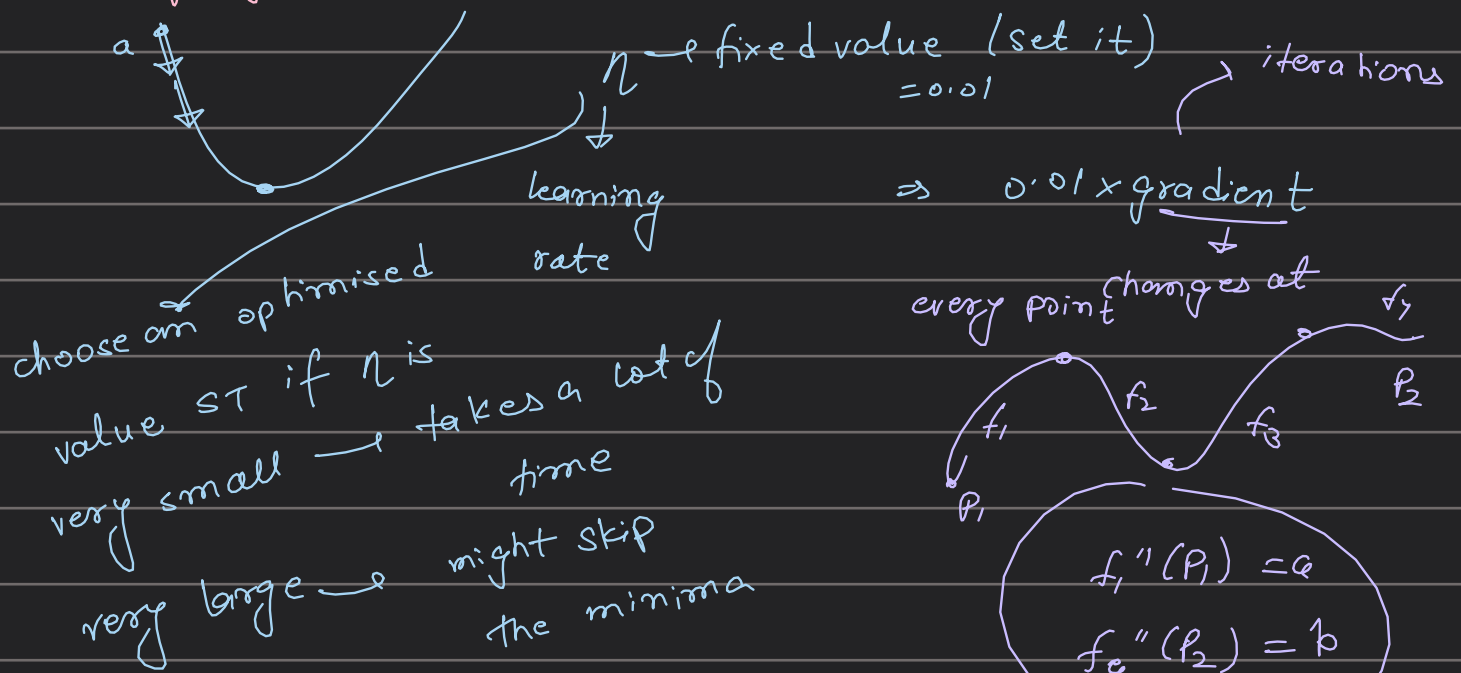
min

Gradient descent:

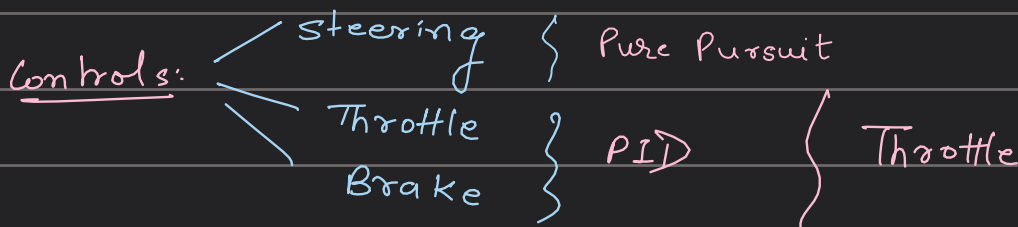
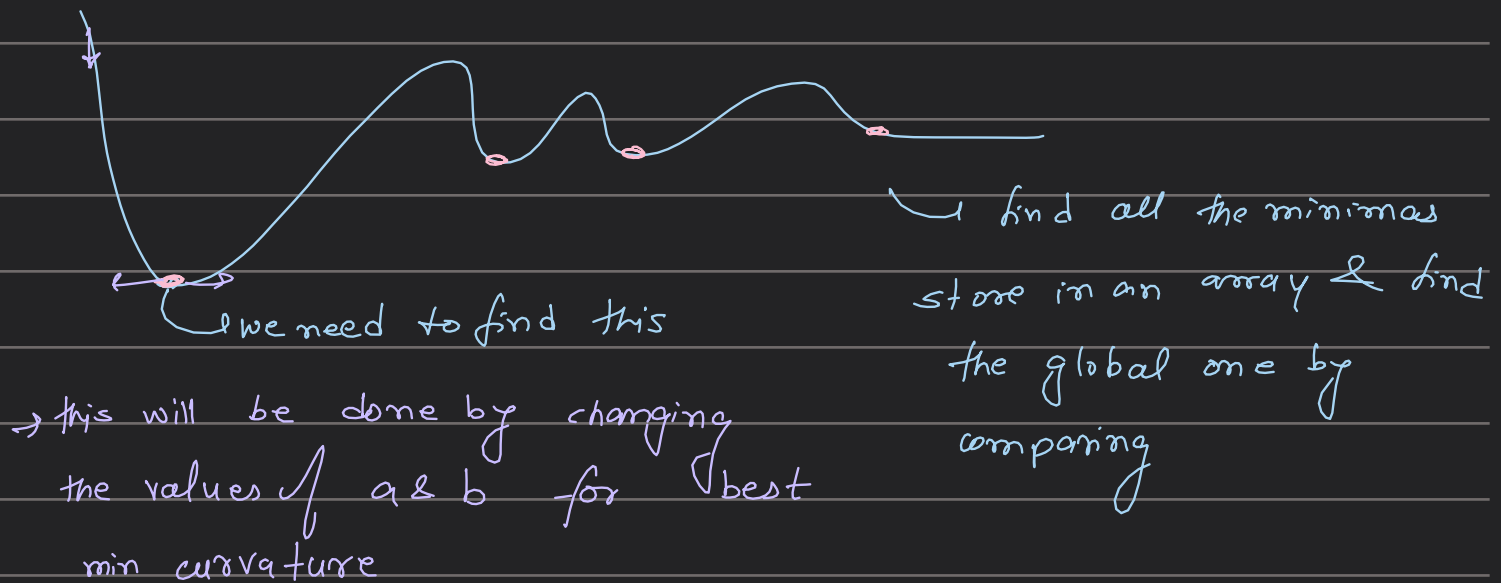
$$K_0(t) = \left| \frac{x''(t)y'(t) - y''(t)x'(t)}{((x'(t))^2 + (y'(t))^2)^{3/2}} \right|$$

total curvature $K(t) = |K_1(t)| + |K_2(t)| + |K_3(t)| + \dots$ minimise this

Method of gradient descent

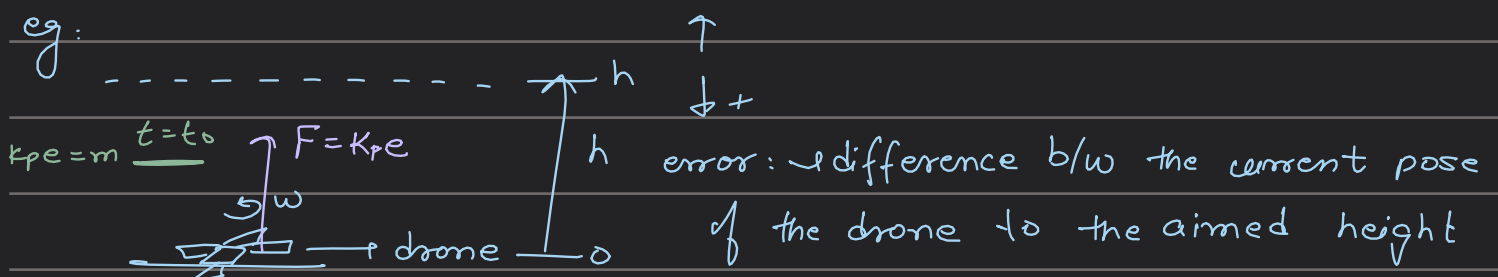


local vs global minima:

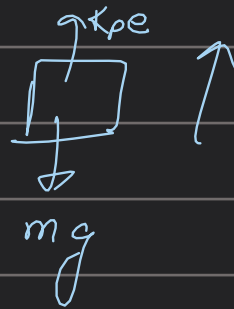


Meet-3

PID: Proportional integral derivative:



$t=0$ with time error dec
 $\text{error} = h - 0 = h$
 (e)



apply a force proportional to error

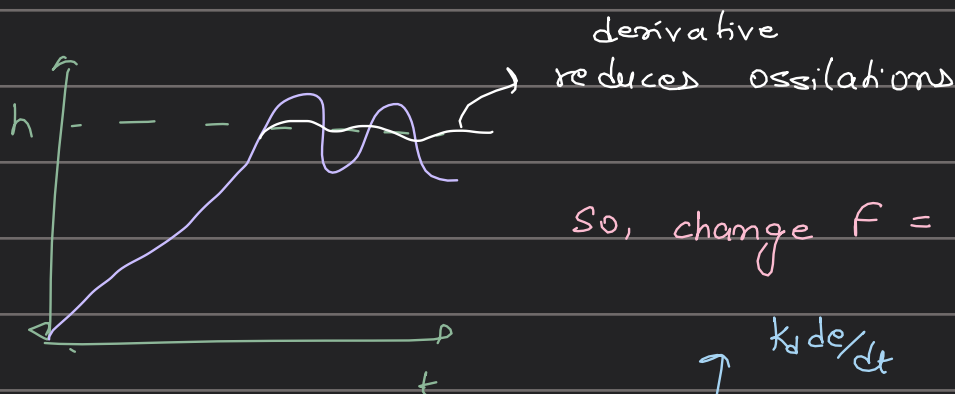
at the equilibrium the error will

$$F_{\text{net}} = k_p e - mg$$

become const but we want to reduce the error

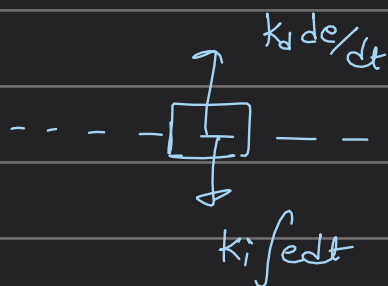
so change $F = k_p e + k_i \int e dt$ it will make the drone reach h

When the drone reaches h $k_p e + k_i \int e dt \neq 0$ so when the drone moves upwards $e \rightarrow -ve \Rightarrow F_{\text{net}} \rightarrow \text{down}$



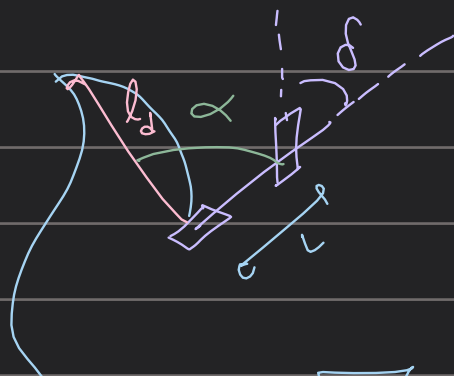
oscillations takes place

So, change $F = k_p e + k_i \int e dt - k_d \frac{de}{dt}$



the integral & derivative terms will balance each other out & thus the oscillations will die

Steering:
 (Pure pursuit)



$$\delta = f(l_d, L, \alpha)$$

we have to fixed tune l_d

speed of the car

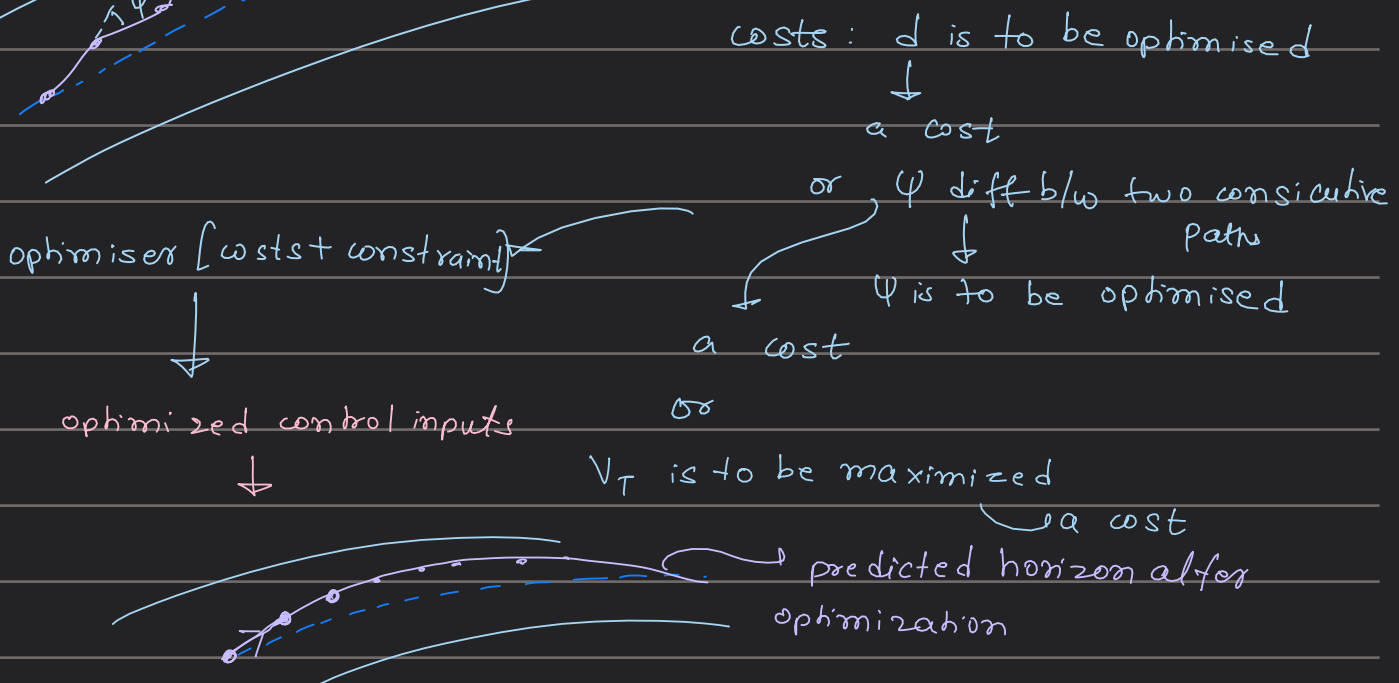
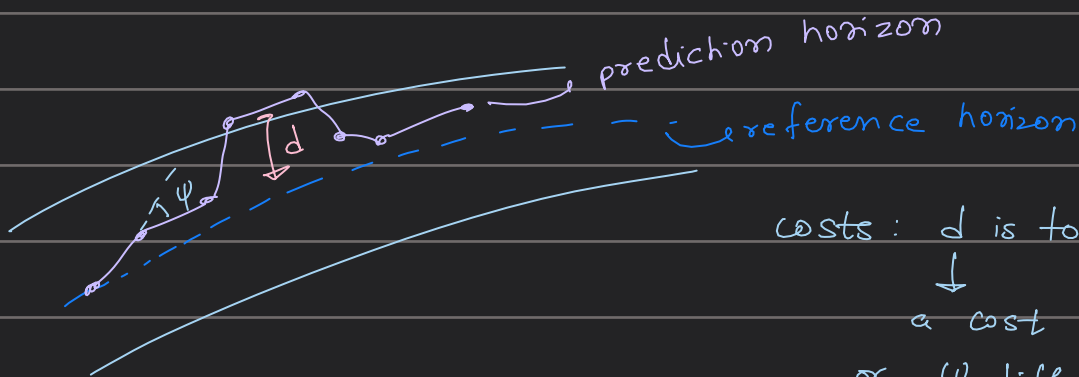
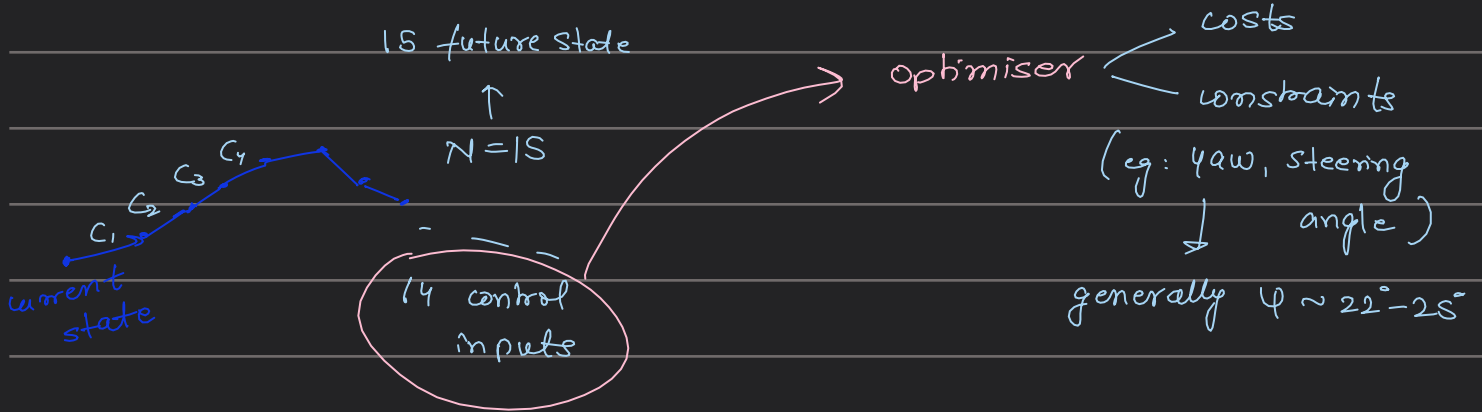
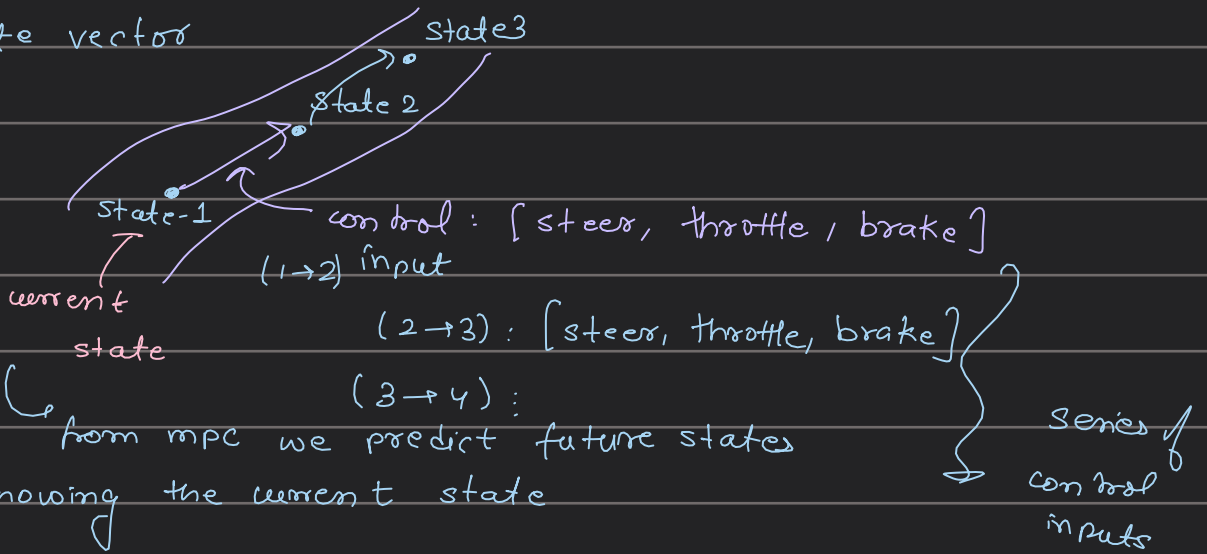
$$l_d = l_{d \text{ min}} + v K$$



MPC - Model Predictive Control

$$\begin{bmatrix} x \\ y \\ \psi \\ \dot{x} \\ \dot{y} \\ \dot{\psi} \end{bmatrix}$$

state vector

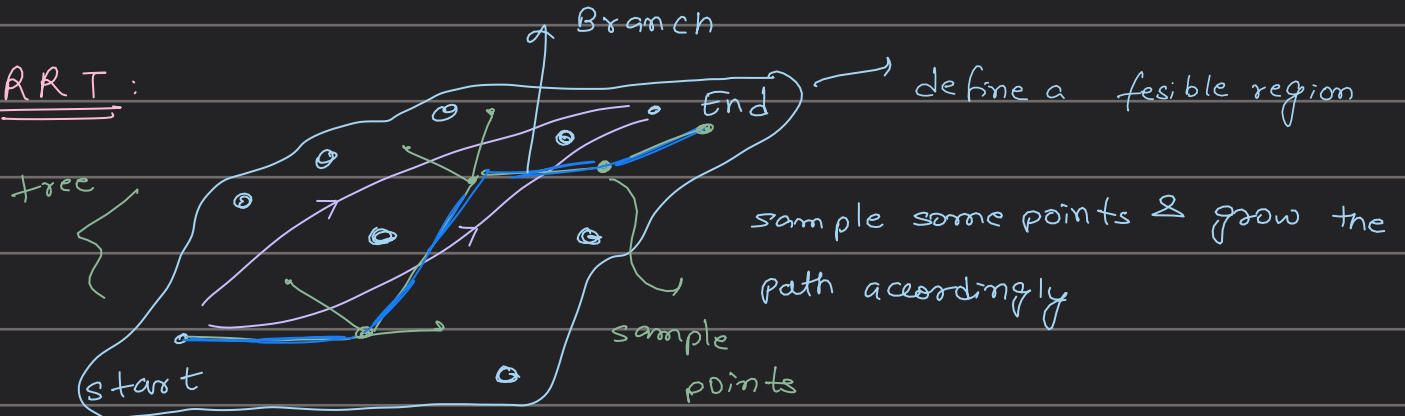


flow:

predict the path $\rightarrow$ control inputs $\rightarrow$ optimizer $\rightarrow$ optimized control inputs $\rightarrow$ better path

this step will continue since after each path the reference horizon changes thus the prediction horizon also does.

RRT:



- delete the growth which leads you to an obstacle
- from all the growth which finally leads to the end

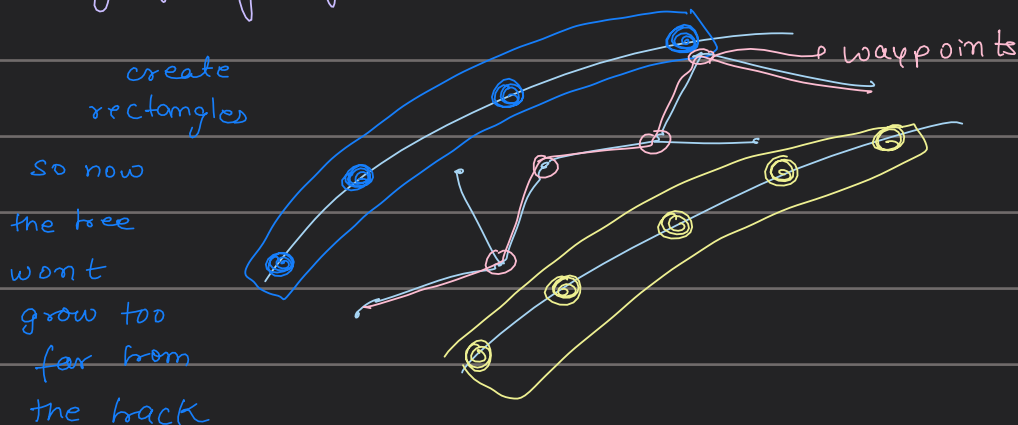
$\Rightarrow$ will select the **branch** st the cost fxn is optimized
for branch selection (here we don't need the colours of the cones)

cost fxn:

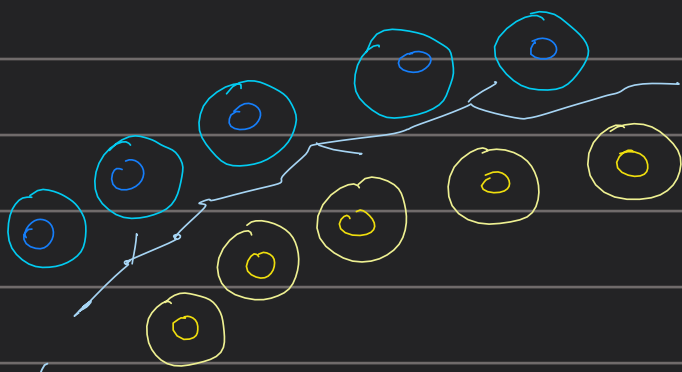
- no. of cones each side of the growth
- length of the branch

(using colours)

using **greedy algorithm** we can sort blue & yellow cones



because of the colors the obstacle becomes better
 $\downarrow$
less random growth of the tree



due to symmetry the tree
will grow along the track

sample each of the
obstacle instead of
sampling all the blue
& yellow cones together

↓
this will also
prevent random points
in the back from
getting into the sample